INTERVIEW FIELD NOTES

# 字节跳动

AI Infra / RL 推训一致性与性能优化

## 从语音转写到可复用的面试能力地图

以项目真实性为第一筛选条件：要求把 RL 推训一致性、MoE 路由、GPU 性能优化和系统稳定性讲成可验证的工程闭环，并穿插算法编码。

| | |
|---|---|
| **整理口径** | 2 份多轮记录综合。内容经语义去噪、合并与重写，不保留逐字稿。 |
| **隐私处理** | 已剔除候选人身份、教育/履历、地点、时间、个人选择、流程沟通与全部反问；不引用原始对话。 |
| **阅读方式** | 先看考察地图定位重点，再按题型练习“定义 - 原理 - 方案 - 验收 - 边界”的闭环表达。 |

### 通用高分闭环

任何技术或业务题，都不要直接跳到“方案”。先固定问题边界和度量方式，再展现分析过程、取舍和验收证据。

定义指标 › 复现偏差 › 对齐路径 › 性能优化 › 回归守护

小红书小鸿别跳蚤

# 01 考察地图

下图为材料中反复出现的主题的相对覆盖度，用于决定复习优先级，而非量化评分。




## 面试官真正想验证什么？

### 问法特征

同一个项目会反复改写提问：先问原理，再问指标，再问异常现象，最后追问为何不是其他原因。必须避免只背项目结论。

### 一致性主线

重点是采样/回放、训练执行与路由决策之间的偏差如何被测量、归因和控制。任何指标都要说清分子、分母及低/高分别代表什么。

### 性能主线

小 shape、decode、路由 GEMM 等场景应从工作量、访存、并行度和 launch 开销共同判断，不可只用单一吞吐结论。

### 工程主线

对图捕获、通信和融合算子交互引发的问题，强调复现矩阵、二分缩小与回归测试，而非只描述修了某个开关。

小红书小鸡别被卖

## 答题表达式

> 问题定义（场景、约束、指标） -> 机制拆解（数据流、资源、依赖） -> 方案选择（备选与取舍） -> 验证（正确性、性能、稳定性） -> 边界（何时无效、如何回退）。

# 02 高频题型与回答框架

以下不是原始问答转写，而是把多轮追问归纳为可迁移的答题骨架。练习时请替换为可公开、不可识别个人信息的案例。

| 题型 / 考察意图 | 可复用回答框架 |
| --- | --- |
| **如何讲清 MoE 路由回放的一致性?**<br>考察能否把论文/系统机制转为工程指标。 | 先界定收集点、回放点与比较对象；给出路由不一致率等指标的定义；说明偏差会如何影响训练目标；再讲检测、对齐和长轮验证，区分“相关性”与“因果修复”。 |
| **Decode 小矩阵为什么跑不满?**<br>考察 roofline、并行粒度与 kernel 意识。 | 先判断问题是算力、带宽还是 launch/同步主导；检查 tile、并发 CTA、数据布局与访存合并；分别评估融合、专用 kernel、分组执行等路径，并以端到端延迟验收。 |
| **CUDA Graph 与并行通信同时启用为何有风险?**<br>考察执行时序和资源生命周期。 | 说明 capture/replay 对 shape、指针、stream 和通信顺序的约束；通过最小配置复现并逐项放回优化；修复后覆盖不同并行度、输入与长轮运行，确保无死锁/超时。 |
| **算法题如何兼顾思路与工程质量?**<br>考察基本功与沟通。 | 先澄清输入、边界和复杂度目标；给出核心数据结构与不变量；用样例走读；最后补充时间/空间复杂度与极端 case。代码完成后主动解释测试覆盖。 |

### 项目题的五层追问

背景与目标 › 瓶颈证据 › 方案与取舍 › 结果与验证 › 复盘与边界

同一个项目至少准备三层深度：90 秒概览、3 分钟机制解释、10 分钟证据链。若缺少真实数据，宁可说明观测方法和待验证假设，也不要编造结论。

小红书小鸽别被骗

# 03 面试前 30 分钟清单

| | |
|---|---|
| 01 | 能在 90 秒内定义每个关键一致性指标。 |
| 02 | 准备一个“先禁用、后定位、再修复”的稳定性案例模板。 |
| 03 | 复习 MoE 的 top-k、负载均衡、EP 通信和 token dispatch。 |
| 04 | 手写常见哈希、双指针、栈/队列题，并练习边界说明。 |

### 模拟练习规则

随机抽取一题，先用 20 秒复述问题与边界；再用 90 秒给出结构化答案；最后接受三轮“为什么 / 如何验证 / 何时失效”的追问。录音回听时，只检查逻辑、术语准确性和证据完整性，不保留个人信息。

本讲义只输出可复用的岗位知识与面试方法，不含个人隐私、逐字转写或反问内容。

小红书小鸿别放弃

INTERVIEW FIELD NOTES

# 字节跳动

大模型推训优化 / Kernel 与量化

## 从语音转写到可复用的面试能力地图

强调把高性能优化讲透：从模型链路和硬件特性出发，解释算子、量化、图执行与并行策略如何共同影响训练/推理成本。

| | |
|---|---|
| **整理口径** | 1 份记录提炼。内容经语义去噪、合并与重写，不保留逐字稿。 |
| **隐私处理** | 已剔除候选人身份、教育/履历、地点、时间、个人选择、流程沟通与全部反问；不引用原始对话。 |
| **阅读方式** | 先看考察地图定位重点，再按题型练习“定义 - 原理 - 方案 - 验收 - 边界”的闭环表达。 |

### 通用高分闭环

任何技术或业务题，都不要直接跳到“方案”。先固定问题边界和度量方式，再展现分析过程、取舍和验收证据。

**业务形状** › **硬件瓶颈** › **实现策略** › **精度约束** › **端到端验收**

小红书小鸿别贩卖

# 01 考察地图

下图为材料中反复出现的主题的相对覆盖度，用于决定复习优先级，而非量化评分。




## 面试官真正想验证什么？

### 核心要求

回答要同时有“上层业务形状”和“下层执行事实”。例如，先解释 prefill/decode 差别，再推导 kernel 设计与图执行的收益边界。

### 量化要求

不仅问格式定义，还会问 scale、离群值、权重/激活选择、精度回退与线上监控。不能把 FP8、INT8 和权重量化简单等同。

### 性能要求

区分单 kernel、子图与端到端指标；明确 warmup、输入分布和硬件/软件版本，避免不可复现的绝对结论。

### 表达要求

项目成果必须让面试官能追问：瓶颈证据是什么、方案为何可行、收益从哪来、哪些场景不会收益。

小红书小鸡别放弃

## 答题表达式

> 问题定义（场景、约束、指标） -> 机制拆解（数据流、资源、依赖） -> 方案选择（备选与取舍） -> 验证（正确性、性能、稳定性） -> 边界（何时无效、如何回退）。

# 02 高频题型与回答框架

以下不是原始问答转写，而是把多轮追问归纳为可迁移的答题骨架。练习时请替换为可公开、不可识别个人信息的案例。

| 题型 / 考察意图 | 可复用回答框架 |
| --- | --- |
| **算子融合的收益与代价？**<br>考察是否理解 launch、访存与可维护性的平衡。 | 说明减少中间读写和 launch 的收益；再分析寄存器压力、occupancy、shape 泛化、调试复杂度；给出按热点和形状分桶启用、保留 fallback 的工程策略。 |
| **INT8 / FP8 量化如何落地？**<br>考察数值表示和工程闭环。 | 先区分数值范围与精度特性；定义 scale 的统计粒度与更新时机；分析离群值和累积误差；最后以离线精度、线上质量、延迟和显存共同决策。 |
| **如何评估 CUDA Graph 是否值得接入？**<br>考察静态图约束与收益边界。 | 估算 launch 占比和 shape 稳定性；说明 warmup、capture、replay 和内存地址约束；对动态输入使用分桶或 fallback；以 p50/p99 与成功率共同评估。 |
| **优化如何避免“局部更快、整体更慢”？**<br>考察系统视角。 | 建立从请求到输出的时间分解；先消除全局关键路径，再看局部热点；测量同步、通信、显存和队列效应；在真实工作负载上做 A/B 与回归。 |

### 项目题的五层追问

背景与目标 › 瓶颈证据 › 方案与取舍 › 结果与验证 › 复盘与边界

同一个项目至少准备三层深度：90 秒概览、3 分钟机制解释、10 分钟证据链。若缺少真实数据，宁可说明观测方法和待验证假设，也不要编造结论。

小红书小鸡别放弃

# 03 面试前 30 分钟清单

| | |
|---|---|
| 01 | 复习量化格式、scale 粒度、校准与回退机制。 |
| 02 | 为一个 kernel 优化准备 roofline/trace 解释。 |
| 03 | 能清晰对比 prefill 与 decode 的资源主导项。 |
| 04 | 准备 3 个常见性能指标：吞吐、TTFT、ITL/p99。 |

### 模拟练习规则

随机抽取一题，先用 20 秒复述问题与边界；再用 90 秒给出结构化答案；最后接受三轮“为什么 / 如何验证 / 何时失效”的追问。录音回听时，只检查逻辑、术语准确性和证据完整性，不保留个人信息。

---

本讲义只输出可复用的岗位知识与面试方法，不含个人隐私、逐字转写或反问内容。

小红书小鸿别放弃

# 字节跳动三面：通信稳定性与算子基准

> 技术深挖复盘 · CUDA Graph / TP / 融合通信 / MoE / Benchmark。根据个人转写改写，公司与轮次沿用来源文件标注，不代表官方题库。

## 一、这一轮主要考什么

核心是稳定性问题的定位过程，以及算子开发与模型、硬件知识的连接。故障修复不仅要描述“改了什么”，还需要解释版本、数据类型、同步机制和最小回归。

## 二、源记录中可确认的提问

1. Graph 开启且 TP 大于一时出现 hang，怎样设计开关和卡数对照？
2. 如何将问题进一步缩小到某个融合 All-Reduce / 归一化算子？
3. 数据完成标记放在哪里、由谁写入、怎样轮询和更新？
4. 为什么原实现使用浮点载荷或特殊值作为完成标记？设计边界是什么？
5. 位模式判断、数据类型和等待条件之间怎样对应？
6. 修复后做了哪些两卡和模型级 Graph 开关回归？
7. 为什么选择 CUDA、Triton 或 CuTe DSL？如何考虑可维护性和协作？
8. MoE 推训路由不一致具体表现在哪里，路由回放记录和复用什么？
9. 一个 token 通过 MoE 的前向流程是什么？有哪些计算与通信步骤？
10. 确定性 dQ 路径的瓶颈、分块和依赖怎样解释？
11. 算子 benchmark 怎样组织 shape、dtype、batch 和多种实现？
12. 怎样输出可复用的测试结果，而非手工汇报一个加速倍数？
13. 评价某种 GPU 时，哪些结论来自规格、哪些来自自己的实测？
14. 下一步怎样补硬件细节和通信方面的理解？

## 三、整理补充：稳定性案例的完整结构

### 从现象收敛到路径

制作实验矩阵，列出 Graph 开关、TP 卡数、目标融合路径开关、输入类型、版本和是否复现。每次只改变明确因素，保留“没有复现”对应的测试时长和次数。不能把一次成功运行当成根因已经证明。

### 从路径收敛到协议

画两侧写入、轮询、完成和复用缓冲区的状态图。将载荷位宽、哨兵位模式、比较宽度和状态发布顺序逐一对应到代码。原记录存在负零、十六位/三十二位与常量的识别歧义，本篇不根据口述给出具体十六进制修复值。

### 从补丁走到回归

先构造能触发等待错误的最小用例，再做不同类型、卡数、Graph 模式和重复回放的回归；明确修复的是哪个项目版本或内部派生版本。不要把某个库在特定调用链中的融合路径，描述成 CUDA Graph 通用地自动融合算子。

## 四、算子基准如何准备

准备统一输入表，包含 shape、layout、dtype、随机种子、误差标准、预热和计时范围。分别比较手写实现、计算库实现与原框架路径，记录版本及选中算法。表中每一项收益都能回到具体用例，而非只给平均值。

硬件比较还应固定软件、工作负载与计时方法。对未经公开规格或同口径测试核对的制程、缓存层级、带宽及相对性能，只列为待研究问题，不写成产品评价结论。

## 五、来源与范围

来源：`字节 AI INFRA 三面 (transcribed on 14-Sep-2026 22-12-45).txt`。原文件部分文字以单字空格分隔，已按上下文整理。删除身份、同事姓名、内部组织和模型信息、薪酬与流程；未收录未核实的硬件对比结论。与已有综合复盘不同，本篇集中补充通信故障收敛与基准测试的连续追问。